

Towards Understanding Docker Build Faults in Practice: Symptoms, Root Causes, and Fix Patterns

YIWEN WU*, National University of Defense Technology, China and State Key Laboratory of Complex & Critical Software Environment, China

YANG ZHANG*[†], National University of Defense Technology, China and State Key Laboratory of Complex & Critical Software Environment, China

TAO WANG[†], National University of Defense Technology, China and State Key Laboratory of Complex & Critical Software Environment, China

BO DING, National University of Defense Technology, China and State Key Laboratory of Complex & Critical Software Environment, China

HUAIMIN WANG, National University of Defense Technology, China and State Key Laboratory of Complex & Critical Software Environment, China

Docker building is a critical component of containerization in modern software development, automating the process of packaging and converting sources into container images. It is not uncommon to find that Docker build faults (DBFs) occur frequently across Docker-based projects, inducing non-negligible costs in development activities. DBF resolution is a challenging problem and previous studies have demonstrated that developers spend non-trivial time in resolving encountered build faults. However, the characteristics of DBFs are still under-investigated, hindering practical solutions to build management in the Docker community. In this paper, to bridge this gap, we present a comprehensive study for understanding the real-world DBFs in practice. We collect and analyze a DBF dataset of 255 issues and 219 posts from GitHub, Stack Overflow, and Docker Forum. We investigate and construct characteristic taxonomies for the DBFs, including 15 symptoms, 23 root causes, and 35 fix patterns. Moreover, we study the fault distributions of symptoms and root causes, in terms of the different build types, i.e., Dockerfile builds and Docker-compose builds. Based on the results, we provide actionable implications and develop a knowledge-based application, which can potentially facilitate research and assist developers in improving the Docker build management.

CCS Concepts: • **Software and its engineering** → **Software configuration management and version control systems**; **Software maintenance tools**; **Software post-development issues**.

Additional Key Words and Phrases: Docker build, Build fault, Empirical study

*Yiwen Wu and Yang Zhang made equal contributions to this work.

[†]Corresponding authors.

Authors' Contact Information: [Yiwen Wu](#), National University of Defense Technology, Changsha, China and State Key Laboratory of Complex & Critical Software Environment, Changsha, China, wuyiwen14@nudt.edu.cn; [Yang Zhang](#), National University of Defense Technology, Changsha, China and State Key Laboratory of Complex & Critical Software Environment, Changsha, China, yangzhang15@nudt.edu.cn; [Tao Wang](#), National University of Defense Technology, Changsha, China and State Key Laboratory of Complex & Critical Software Environment, Changsha, China, wangtao2005@nudt.edu.cn; [Bo Ding](#), National University of Defense Technology, Changsha, China and State Key Laboratory of Complex & Critical Software Environment, Changsha, China, dingbo@nudt.edu.cn; [Huaimin Wang](#), National University of Defense Technology, Changsha, China and State Key Laboratory of Complex & Critical Software Environment, Changsha, China, hmwang@nudt.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2994-970X/2025/7-ARTFSE040

<https://doi.org/10.1145/3715757>

ACM Reference Format:

Yiwen Wu, Yang Zhang, Tao Wang, Bo Ding, and Huaimin Wang. 2025. Towards Understanding Docker Build Faults in Practice: Symptoms, Root Causes, and Fix Patterns. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE040 (July 2025), 23 pages. <https://doi.org/10.1145/3715757>

1 Introduction

As a popular containerization technology, Docker has been increasingly utilized in software development [Cito et al. 2016, 2017; Mouat 2015; Shalaev 2021], and is becoming a critical component in the CI/CD pipeline [Anderson 2015; Atkinson and Edwards 2018; Garg and Garg 2019; Zhang et al. 2018]. Docker extends Linux Containers (LXC) [Dua et al. 2014] with higher-level APIs and additional functionality. An application encapsulated by Docker is distributed in the form of a *Docker image*. Developers can easily build and deploy applications based on a large number of publicly reusable Docker images [Ksontini et al. 2021]. Recently, Docker images have been adopted by 20M+ monthly developers and obtained 20B+ monthly downloads¹. In the 2023 survey of the Stack Overflow [Stack-Overflow 2023], Docker is ranked as the first place in “Top-used other tools” with the participation of 90,000+ developers.

In the containerization process, the Docker building plays a crucial role in packaging and transforming the sources into container images. When a Docker build is triggered either automatically or manually, the build server will clone the source code and configuration files, install dependencies, produce Docker images, and report results to the developers. For a single container application, all the needed information about the environment and program execution is specified by the *Dockerfile*. The Docker-compose [Docker 2022b] is a tool for building multi-container applications on Docker and it is defined by a *docker-compose.yaml* (or *docker-compose.yml*) to orchestrate their execution. Docker builds demonstrate multiple benefits to software development, such as automated deployment, deterministic workflow, reliable production, and fast releasing, ultimately benefiting both developers and end-users [Combe et al. 2016; Mouat 2015; Xu and Marinov 2018].

Accompanying the aforementioned benefits, however, developers may encounter various build problems in preparing, executing, and maintaining builds in the Docker-based software development. We refer to these problems as **Docker build faults (DBFs)**. For example, we find many developers complain that Docker build failed [Pprathima3 2018], hanged [Mike 2020], or had poor performance [R. 2017] in practice. Also, Cito et al. [2017] found a 34% breakage in Docker builds from a sample of 560 GitHub projects. Such DBFs can hurt developers’ experience, consume resolution effort, and slow down development efficiency. For example, Wu et al. [2020] found that the median fix time of Docker broken builds is 44.2 minutes (in comparison, developers spend a median of 5 and 12 minutes fixing C++ and Java build failures, respectively [Seo et al. 2014]). Therefore, to provide developers and researchers with practical guidelines on the DBF resolution, it is necessary to study the characteristics of real-world DBFs.

Several recent studies have shed some light on this problem. Prior works [Cito et al. 2017; Wu et al. 2020] provided the basic statistics for Docker build failures, but they did not analyze other categories of DBFs; Haque et al. [2020] analyzed the challenges faced by Docker developers, but they focused on broader concepts of Docker technology and lack analysis for build faults. Henkel et al. [2021] studied limited (only 4) root causes of Dockerfile-related build failures, and did not give detailed analysis on other root cause types; and Wu et al. [2022] investigated the severity of the slow build problem faced by Docker developers, and only partially inspected the root causes. In summary, those studies focused only on the broader Docker technology, or targeted at only a specific build fault type, leaving many other categories of DBFs unexplored. There remains

¹<https://www.docker.com>, Accessed September 12, 2024.

a limitation in the comprehensive understanding of the full spectrum of DBF types, their root causes, and remediation methods. This gap can lead to non-negligible resolution costs and hinder the efficiency of containerized workflow, emphasizing the need for more extensive analysis and community-driven solutions to improve the robustness of Docker build management.

To fill this knowledge gap, in this paper, we present the first comprehensive study for understanding the real-world DBFs in practice. We collect a DBF dataset of 255 issues and 219 posts from three data sources: GitHub² (the largest public code repository host), Stack Overflow³ (one of the most popular Q&A communities), and Docker Forum⁴ (the official forum for Docker). Based on the dataset, we manually summarize DBF symptoms, root causes, and fix patterns. Our study focuses on three questions that we believe could provide insights for developers and researchers.

RQ1 (Symptoms): *What are the symptoms of DBFs in practice?* This RQ categorizes the symptoms or effects of different DBFs. Understanding the symptoms can help developers assess the impact of DBFs and triage them appropriately.

RQ2 (Root causes): *What are the root causes of DBFs?* To understand the nature of DBFs, it is critical to identify the root cause, which helps developers explore potential approaches for avoiding and fixing faults. This RQ examines the detailed root causes of DBFs among the derived symptoms.

RQ3 (Fix patterns): *What are the fix patterns of DBFs?* Fix patterns provide principled ways to resolve DBFs. We study the accepted answers, comments, and patches of each fault to identify its fix method. Common fix methods across multiple faults are grouped into different fix patterns.

Through these analyses, we summarize 15 symptoms, 23 root causes, and 35 fix patterns. In addition, we study the distribution of symptoms and root causes on the different build types, i.e., Dockerfile builds and Docker-compose builds; the root cause distribution for different symptoms, and the fix pattern distribution for different root causes. The classification (i.e., symptoms, root causes, and fix patterns) and distribution results can help developers and researchers better understand the various DBFs and their characteristics, providing actionable implications for improving the Docker build management. Furthermore, we design and implement a knowledge-based application, DBF-ASSISTANT, to assist developers in tackling DBFs with our accumulated knowledge.

In summary, this paper makes the following contributions:

- To the best of our knowledge, this is the *first* comprehensive study for understanding the characteristics of faults in Docker builds. We construct a dataset of 474 DBFs from three representative data sources, including GitHub, Stack Overflow, and Docker Forum, which serves as the basis of our study as well as a valuable benchmark to foster future research.
- We investigate the DBFs from different perspectives (i.e., symptoms, root causes, and fix patterns), construct the taxonomies, and distill several findings.
- We provide a series of implications based on our findings, which help shed light on tackling DBFs in Docker-based software development, and also develop a knowledge-based application, named DBF-ASSISTANT, to help developers in understanding and fixing DBFs.

2 Docker Build

Docker is an OSS project implementing operating system-level virtualization, which builds on many technologies from operating systems research: LXC [Dua et al. 2014] (Linux Containers), virtualization of the OS [Boettiger 2015], etc. The technology is primarily intended for developers to create and publish containers [Anderson 2015]. Docker launches its containers from a Docker image, which is a series of data layers on top of a base image [Anderson 2015; Zhang et al. 2022], e.g.,

²<https://github.com/>

³<https://stackoverflow.com/>

⁴<https://forums.docker.com/>

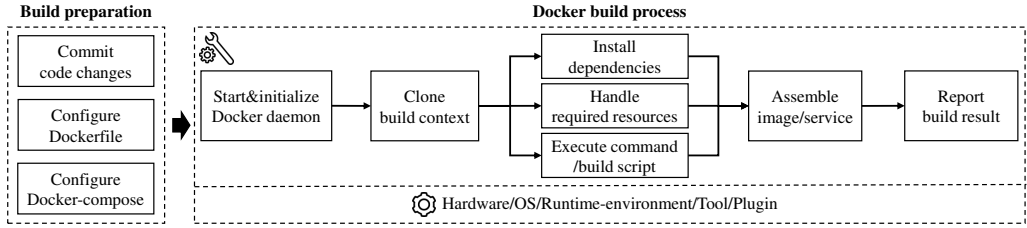


Fig. 1. Docker build process.

ubuntu. With containers, applications can share the same operating system and, whenever possible, libraries and binaries [Bernstein 2014]. The content of the container is defined by the declarations in the **Dockerfile** [Cito et al. 2017] which specifies the Docker commands and the order of their execution. In addition to Dockerfile (i.e., single container application), Docker also supports the use of **Docker-compose** [Docker 2022b], which is a tool for defining and building multi-container applications. With Docker-compose, developers can use a YAML file (named *docker-compose.yaml* or *docker-compose.yml*) to configure application’s services [Ibrahim et al. 2021], orchestrating the creation and management of the containers [Smith 2017].

The ease of packaging and deploying applications is a major reason why containers have become popular [Du et al. 2017]. Thus, executing the build process (i.e., building) is crucial to the containerized workflow. Fig. 1 depicts the main phases of the Docker build process. Before building, developers need to prepare the relevant source code, Dockerfile or Docker-compose configuration. Subsequently, the build process can be triggered either automatically or manually. There are two types of trigger commands for Docker builds: (1) “docker build” command: it builds a single container application from a Dockerfile and a *context* [Docker 2022a] (we refer to it as **Dockerfile build**), and (2) “docker-compose build” command: it builds a multi-container application from a *docker-compose.yml* and a *context* (we refer to it as **Docker-compose build**). The context is the set of files located in the PATH where *Dockerfile* or *docker-compose.yml* resides. Then, the build process starts by starting and initializing the Docker daemon [Whoibrar 2024] (A background service that runs on a host system and manages containers). Next, it clones the configuration and context. After installing all the required dependencies, handling required resources, and executing command/build script, the image is built or the service is assembled. Finally, if all the previous phases are successful, the image/service is created and published, and the developer is notified with the results. If any of the phases fails, the build process stops and reports error messages.

3 Methodology

We first describe our methodology for collecting and analyzing DBFs, as shown in Fig. 2.

3.1 Data Collection

To obtain a comprehensive understanding of DBFs, we use three representative data sources for our analysis: (1) GitHub (GH) issues, (2) Stack Overflow (SO) posts, and (3) Docker Forum (DF) posts. We chose GH and SO for our analysis because: they are widely used platforms, hosting a broad spectrum of problems raised by developers engaged in real-world development tasks; they are likely to contain problems about the DBFs due to their diversity. Prior work has shown that studying GH and SO can give significant insights into software developers’ concerns about a technology [Bagherzadeh and Khatchadourian 2019; Bajaj et al. 2014; Haque et al. 2020; Rosen and Shihab 2016; Zhang et al. 2024]. In addition to GH and SO, we also consider DF as a data source. Because DF is an official and specialized platform where Docker developers can discuss and troubleshoot Docker-specific problems.

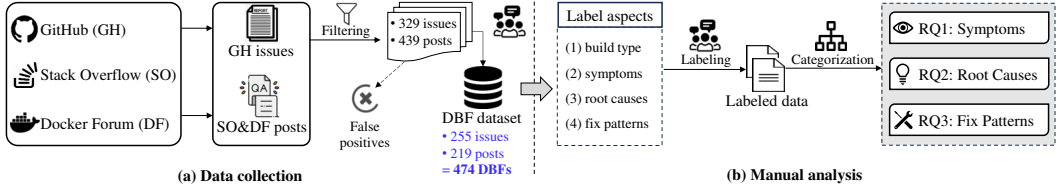


Fig. 2. Overview of our study, including two major steps: (a) Data collection and (b) Manual analysis.

Collecting GH issues. We use the GitHub Search API [GitHub 2022] to obtain the DBF-related issues. Specifically, we search issues with titles containing $keyword \in \{“docker build”, “docker-compose build”\}$, and extract issues labeled with $label \in \{bug, defect, error, type:bug, type:defect, type:error\}$ to exclude those about feature request or document-related problems. The query is conducted on January 1, 2024. This yields 3,781 issues from 2,622 repositories. Then, we only consider the issues that are already *closed* by developers because those issues have been resolved by developers and likely contain adequate information for bug analysis. Kalliamvakou et al. [2016] pointed out that there is a risk of including inactive or abandoned repositories when working with GH data. To mitigate this risk, we follow prior work [Bühmann and Ghafari 2022; Li et al. 2021] and only consider non-forked repositories that have ≥ 100 commits, ≥ 10 stars, and ≥ 10 forks. In this way, we obtain 861 issues from 648 repositories. Based on these issues, we manually examine each of them to exclude invalid ones, e.g., *issues without description, not written in English, or closed with ‘not planned’*, etc. Finally, our issue dataset contains **329 candidate issues** from 267 repositories.

Collecting SO&DF posts. We collect post data through two data sources, SO and DF. First, we obtain and extract the SO post data leveraging the Stack Exchange Data Explorer [Stack-Overflow 2024], which provides up-to-date access to the latest data dump by SO. We identify all DBF-related posts by checking if they have tag *“docker-build”* OR with titles containing $keyword \in \{“docker build”, “docker-compose build”\}$. The query is also made on January 1, 2024, which obtains 1,508 posts. To ensure the quality of selected posts and to reduce the ambiguity in the following analysis, we only keep the posts with an accepted answer. Besides, we follow prior study [Islam et al. 2019; Ponzanelli et al. 2014] and only consider these high-quality posts (i.e., post score > 0). As a result, we obtain 430 SO posts. Second, to collect DF posts, we search the forum directly, checking for posts with titles containing $keyword \in \{“docker build”, “docker-compose build”\}$, created before January 1, 2024, and have a *“solved”* status. This process gets 28 initial posts. After removing the irrelevant posts, we end up with 9 relevant DF posts. Finally, our post dataset consists of **439 candidate posts**.

Identifying DBFs. We manually verify the candidate issues and posts to filter out *false positives* that are not about DBFs. For instance, in our dataset, some posts raise how-to questions (i.e., requesting help with a specific Docker build task or a basic Docker build knowledge) rather than a DBF encountered (for example, post [Hedefalk 2018]: *“How do I copy variables between stages of multi stage Docker build?”*); and some issues indeed discuss DBF, but their comments or related contents are useless (for example, issue [stefan79 2015] only has a comment *“This is fixed”* without resolution details), which can not be used to understand the characteristics of DBFs. In particular, two annotators (i.e., the first two authors with 4-year experience in Docker) independently investigate each issue and post to identify DBFs. To measure the inter-rater agreement during this independent inspection, we employ the widely used Cohen’s kappa coefficient (k) [Cohen 1960], and it reached 0.93. An arbitrator (i.e., the third author with 7 years of Docker experience) is involved to resolve disagreements. Finally, we identify **474 DBFs** (255 issues from 213 repositories, 210 SO posts, and 9 DF posts). This scale is comparable to previous build fault related studies in other contexts, for example, 465 build issues in Maven, Ant, and Gradle projects [Lou et al. 2020] and 91 build problems in Java projects [Hassan et al. 2017].

3.2 Manual Analysis

To answer our three RQs, we manually label each data in the 474 DBFs from 4 aspects: (1) *build type*, which shows **whether** the build is a Dockerfile build (for single container application) or a Docker-compose build (for multi-container application), (2) *symptoms*, which show **what** the negative effects of DBFs, (3) *root causes*, which explain **why** the DBFs occur, and (4) *fix patterns*, which describe **how** a DBF is resolved. Therefore, each issue/post with a specific DBF encountered by a developer will be labeled with the build type, fault symptom, root cause, and fix pattern.

In specific, the aforementioned two annotators conduct manual labeling on the 474 DBFs. Before the labeling, we conduct a training session based on the Docker official documentation [Docker 2024d] to help annotators familiarize themselves with the key Docker build components and knowledge. During the labeling process, we first randomly sample 50% DBF-related data for pilot labeling. We adopt some general labels of symptoms (e.g., Build failure) and root causes (e.g., Configuration error) from existing work on traditional package build faults [Lou et al. 2020; Sulír and Porubán 2016; Zolfagharinia et al. 2019] as the initial labels. Then, following the open-coding scheme [Seaman 1999], two annotators separately read through all contents of an issue or a post (including title, description, comments, answers, commits, and reference links in discussion) to understand the context, thereby determining the labels of DBFs based on the initial general labels by adding DBF-specific labels on demand. Basically, the build type and symptom of a DBF are determined by analyzing the issue/question's title and description. The root cause and fix pattern for fixing a DBF are inferred from the issue/question description, comments, and the fixing commit or the accepted answer. Note that in cases where a root cause may lead to multiple symptoms (which we found to be rare when labeling), we follow two selection rules: the symptom is specifically and explicitly presented in the title or description, or it's a key or final symptom mentioned in the Docker build log. Then, annotators compare the labeling results during an in-person meeting, summarize a set of initial labels, and construct the taxonomies for symptoms, root causes, and fix patterns by grouping similar codes together into categories. The grouping process is iterative, in which they continuously go back and forth between categories and issues/posts to refine the taxonomies. We introduce the arbitrator to resolve the conflicts, if any. All labels and taxonomies are discussed and finalized by all participants.

Based on the labels and taxonomies generated in the pilot labeling, the annotators independently label the remaining 50% DBF-related data. The issues/posts that cannot be classified into the current taxonomies are labeled with a new category. The Cohen's kappa coefficient k values obtained for symptoms, root causes, and fix patterns are 0.97, 0.87, and 0.84, indicating high confidence of agreement, respectively. After this labeling, the annotators and arbitrator resolve some inconsistencies and further refine the labels and taxonomies. The final labeling results are checked and approved by all authors. Of the 474 DBFs, 396 DBFs (215 issues and 181 posts) are Dockerfile builds and 78 DBFs (40 issues and 38 posts) are Docker-compose builds. The manual effort, involved in our data collection and labeling, requires six person-months.

We also investigate another dataset of 52 DBFs created after January 1, 2024 using the aforementioned data collection and analysis methods. This additional analysis aims to assess if new categories have been introduced since the first study was conducted.

4 Study Results

We present the results of the three research questions and the additional analysis.

4.1 RQ1: Symptoms

4.1.1 Symptom Taxonomy. The taxonomy of DBF symptoms is shown in Fig. 3, which is organized into 5 inner categories and 15 leaf categories.

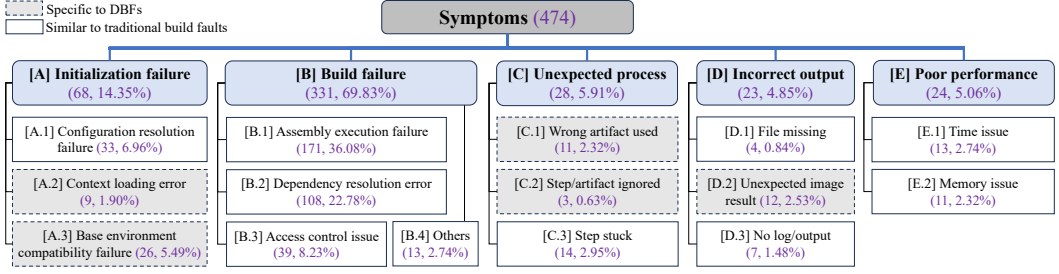


Fig. 3. Taxonomy of symptoms in DBFs.

Initialization failure (A). At the start of executing build, Docker daemon needs to collect the required configuration and context files and initialize the necessary environments. 68 (14.35%) of faults occur in this process, which belong to 3 typical symptoms. Specifically, 33 faults belong to the *Configuration resolution failure* (A.1), e.g., the build argument failed in post P₂₆: “*docker build requires exactly 1 argument - docker build error*”. Also, developers can encounter context loading errors (e.g., in P₂, Docker build fails to load git source code), which contain 9 faults (A.2). We find 26 faults belong to the *Base environment compatibility failure* (A.3). E.g., issue I₁₆₄ reports a failure when building on the “*arm64*” container architecture.

Build failure (B). It means the execution functionality of Docker build is terminated and the Docker build exits irregularly mid-execution by throwing an error, accounting for the largest proportion (69.83%) of DBFs. Note that non-functional terminations like performance issues (e.g., out of memory) are not included in this category. *Assembly execution failure* (B.1) is the most common failure type with 171 faults, referring to that certain assembly commands or bash scripts are not executed successfully. E.g., I₁₀₄ reports executor failed to run at the RUN command stage. Besides, it is an essential step in the Docker build process and is responsible for preparing necessary dependencies directly or transitively specified by developers, which covers 108 DBFs (i.e., B.2). E.g., P₁₅₈: “*Docker ERROR: unsatisfiable constraints when using apk*”. *Access control issue* (B.3) refers to user authorization or permission issues that arise during the build process. E.g., I₁₂₇ reports that Docker build has an error when processing *tar* file since the operation is not permitted. The remaining 13 faults with low frequency (occur only once or twice) are thus categorized in *Others*.

Unexpected process (C). Unexpected process refers to the cases where the process of a certain build task is not as expected although no error is triggered during that build task. This category covers 28/474 (5.91%) faults. Specifically, *Wrong artifact used* (C.1) refers to the use of incorrect data or files in the build process, accounting for 2.32% of the faults. E.g., P₁₈: “*Why aren’t my docker images, built by docker-compose build, using the correct version of my code?*”. There are 3 faults belonging to the *Step/artifact ignored* (C.2) such as P₈₇: “*EXPOSE and CMD are ignored when running docker build*”. Also, 14 faults manifest as getting stuck while performing the steps of a build (C.3). E.g., I₇₃: “*[BUG] Docker build stuck at final 1/2*”.

Incorrect output (D). Some faults do not trigger failures explicitly during the build process, but generate problematic outputs. We group such faults as incorrect output (23/474 faults). Specifically, 4 faults show that Docker build produces containers without required files (D.1), e.g., in P₁₇₃, the developer complains that no files are being added even though it has no build error. Docker build may generate wrong/unexpected image results in 12 faults (D.2), such as P₁₈₂: “*Docker build produces many unnamed images*”. Besides, there are other 7 cases where Docker builds finish without log or output information (D.3). E.g., I₂₁₉: “*Docker build succeeds, but outputs directory empty*”.

Poor performance (E). This is another typical symptom for DBFs, which can slow down the execution processes and consume excessive resources (24/474 faults). *Time issue* (E.1) covers the

performance faults exhibiting high time cost, accounting for the largest portion of poor performance (54.17%). E.g., P_{45} asks “Why docker build is so slow”. Memory issue (E.2) covers 45.83% of the performance faults which consume memory abnormally. We observe that this symptom mainly contains faults related to out of space (e.g., I_{247} : “Docker builds have run out of space”) and invalid memory access (e.g., I_{55} : “Invalid memory access...at address 0x10000001b on Docker build”).

Compared to previous work on traditional package build faults (e.g., Maven [Lou et al. 2020; Sulir and Porubän 2016] and CPAN [Zolfagharinia et al. 2019]), distinct symptoms of the DBFs in our study include A.2, A.3, C.1, C.2, and D.2, which are highlighted in dotted rectangles in Fig. 3. They account for 61 (12.87%) of the 474 DBFs. These differences owe to the fact that Docker builds require a predefined context to be loaded into the Docker daemon first [Anderson 2015], and the output image is closely related to the artifacts used. Additionally, the Docker build system must be consistent with the underlying hardware and system architecture, as Docker images are designed to be portable across environments [Boettiger 2015]. These factors may lead to the distinct symptoms mentioned above.

Finding 1. We construct a DBF symptom taxonomy of 5 inner categories and 15 leaf categories. Build failure is the most common symptom (69.83% of DBFs). Other symptoms include Initialization failure, Unexpected process, Incorrect output, and Poor performance. These wide-ranging impacts motivate the importance of DBFs.

4.1.2 Symptom Distribution. We further investigate the difference of symptom distribution between Dockerfile builds and Docker-compose builds. As shown in Fig. 4, we observe that both Dockerfile and Docker-compose builds have, as the two most frequent symptoms, DBFs that cause build failure or initialization failure (together accounting for more than 80% of DBFs).

While the total frequency of initialization&build failures for the two types of builds is comparably similar, *Initialization failure* is more common in Docker-compose builds (20.51%), than in Dockerfile builds (13.13%). This indicates that the initialization phase of Docker-compose builds is more prone to issues that prevent successful setup. The higher rate of initialization failure could be attributed to the complexity of setting up multiple services and misconfigurations in the *docker-compose.yml* file [Eng et al. 2024], such as incorrect service names and port conflicts. *Incorrect output* is less frequent in Dockerfile builds (4.29%), than in Docker-compose builds (7.69%). This implies that Dockerfile builds are more reliable in terms of producing the expected output. Docker-compose builds show a slightly higher occurrence of poor performance compared to Dockerfile builds. It could be related to the overhead of managing multiple images and services, which can lead to resource contention and low performance. In general, Docker-compose builds might face more complexity due to the orchestration of multiple images/services, which can lead to higher rates of initialization failure and incorrect output. Dockerfile builds might offer more simplicity and control in the single container build process, potentially leading to fewer performance and output issues.

Finding 2. Both Dockerfile builds and Docker-compose builds frequently report build failure and initialization failure as the most frequent DBF symptoms (>80%). In addition, Docker-compose builds are more likely to experience initialization failure and incorrect output, while Dockerfile builds, despite also facing these issues, have a lower incidence of incorrect output and poor performance.

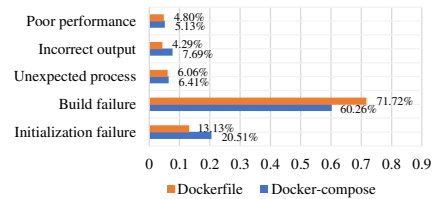


Fig. 4. Comparison of symptom distribution.

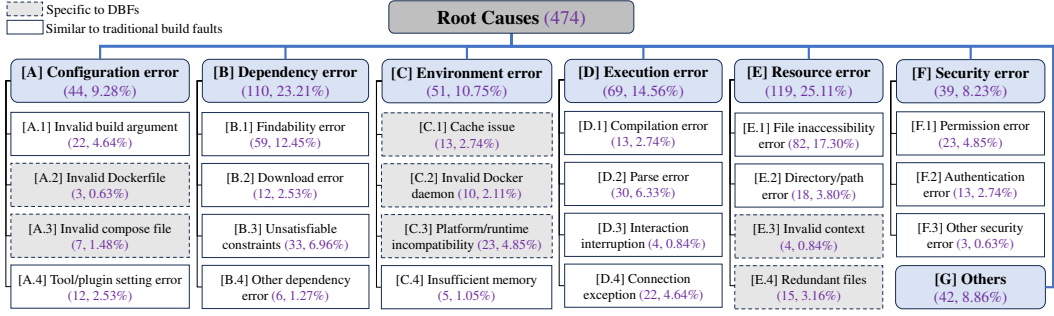


Fig. 5. Taxonomy of root causes in DBFs.

4.2 RQ2: Root Causes

4.2.1 Root Cause Taxonomy. The root cause taxonomies of the studied DBFs are shown in Fig. 5, which is organized into 7 inner categories and 23 leaf categories. The broad scope of root causes indicates the prevalence and diversity of DBFs. There are 42 (8.86%) faults that are difficult to analyze their root causes, and are therefore classified as *Others*.

Configuration error (A). 44/474 (9.28%) faults are caused by the incorrect configuration. In particular, 22 faults are caused by invalid build argument problems. E.g., the fault of P_{50} is caused by not specifying a build context in the build argument. Another 10 faults in this category are due to *Invalid Dockerfile* (e.g., the fault of P_{92} is caused by running an unknown command: ‘EXPAND-ARCHIVE’) and *Invalid compose file* (e.g., P_{120} failed because docker-compose.yml contains an unsupported option: ‘network’). The remaining 12 faults are due to *Tool/plugin setting error*. E.g., the fault of P_{80} is caused by not setting the “- -progress” in the buildkit tool.

Dependency error (B). 110 faults are caused by dependency problems. Specifically, 59 (12.45%) of the faults are caused by *Findability error*, which is the most frequent category in the dependency error, occurs when the required dependencies cannot be found in the specified location (e.g., remote or local repository). E.g., the fault of P_{15} is caused by not finding the required module ‘numpy’. 12 faults are caused by the *Download error* (B.2) triggered in the unsuccessful dependency downloading process. E.g., P_{30} failed due to failure to download required files. *Unsatisfiable constraints* (B.3) refers to developers using the wrong dependency or incorrect dependency version, causing conflicts or incompatibilities in the build process. E.g., P_{61} failed because dependency *vue3* is not compatible with *element-ui* under the base image *node:16-buster-slim*.

Environment error (C). 51 (10.75%) faults are caused by imperfect support of Docker build execution for some hardware/software environments, which can be further divided into 4 leaf categories. Specifically, *Cache issue* is an important environmental root cause, covering 13 faults. E.g., P_{207} encountered a memory issue because of an invalid cache option setting. 10 faults are caused by *Invalid Docker daemon* (C.2) such as P_{202} could not initialize the build environment due to Docker daemon is not accessible. *Platform/runtime incompatibility* (C.3) covers the largest number of environment errors (23/51). E.g., I_{90} ’s developers got no build output because their Docker-compose build does not work in Linux and Mac. Apart from the environmental factors, the *Insufficient memory* (C.4) can also bring faults in some Docker build scenarios (5 faults). E.g., P_{156} had poor build performance due to virtual memory exhaustion.

Execution error (D). This category covers faults caused by specific execution errors, which accounts for 69 (14.56%) of the faults. In particular, *Compilation error* (D.1) refers to the problems that occur during the process of translating source code into container image, accounting for 13 faults. E.g., I_{208} encountered RUN command execution failure because the web component fails to compile. 30 faults are caused by *Parse error* (D.2), i.e., the build scripts are not validated (e.g.,

invalid identifier and undefined properties) in the Docker build process. E.g., P_{140} 's RUN command failed due to an invalid framework identifier in the `.csproj` file. *Interaction interruption (D.3)* leads to 4 faults in this study. E.g., P_4 hanged on the RUN command due to the output pops the message "continuously: Please answer 'yes' or 'no'". The remaining 22 faults are due to the *Connection exception (D.4)*. E.g., the build failure of P_{83} is because it could not connect to `dotnet.testcontainer`.

Resource error (E). Docker build resource introduces 119/474 (25.11%) faults. Particularly, 82 faults are caused by the *File inaccessibility error (E.1)*. E.g., P_9 executing COPY command failed because it could not find files in the local directory. 18 faults are caused by the *Directory/path error (E.2)* such as the fault of I_{20} is caused by wrong directory in `openjdk` configuration. *Invalid context (E.3)* introduces 4 faults. E.g., P_{18} could not build as expected because it used the wrong version of the code as context. The remaining 15 faults are caused by *Redundant files (E.4)*. E.g., I_{247} 's build ran out of space due to not removing useless files.

Security error (F). 39 (8.23%) faults are caused by the root causes in this category. Specifically, 23 faults are caused by *Permission error (F.1)*, e.g., the build failure of P_{198} due to incorrect file permission setting in the COPY command. *Authentication error (F.2)* introduces 13 faults. E.g., the fault of I_{209} is caused by a failure in authenticating GitHub API calls. There are also 3 faults caused by other security errors, e.g., checksum mismatch and signature error.

Compared to studies on traditional package build faults [Lou et al. 2020; Sulír and Porubán 2016; Zolfagharinia et al. 2019], distinct root causes of DBFs include A.2, A.3, C.1, C.2, C.3, E.3, and E.4, accounting for 15.82% (75/474) of the DBFs. It is worth mentioning that although most root causes are similar to prior work, DBFs' root causes can be different since they are more closely aligned with the components (e.g., Docker daemon [Whoibrar 2024] and Docker build cache [Docker 2024b]) and processes (e.g., loading context and container orchestration) specific to Docker builds.

Finding 3. DBFs are caused majorly by Resource error (25.11%) and Dependency error (23.21%). Moreover, Configuration error, Environment error, Execution error, and Security error are common root causes, accounting for other 42.82% of the DBFs.

4.2.2 Root Cause Distribution. We further analyze the relationship between root causes and symptoms, and investigate the root cause distributions in Dockerfile and Docker-compose builds.

Root causes & Symptoms. To understand the relationship between root causes and symptoms, we obtain a distribution mapping from root causes to symptoms, as shown in Fig. 6. We find a complex correlation between root causes and symptoms. On the one hand, we observe that *Dependency error*, *Resource error*, and *Execution error* are the most common root causes of *Build failure* symptom, accounting for 32.33% (107/331), 26.59% (88/331), and 19.34% (64/331) of build failures, respectively. *Initialization failure* are mainly caused by *Configuration error* and *Environment error*, accounting for 47.06% (32/68) and 35.29% (24/68) of initialization failures, respectively. We find that security error is primarily associated with build failure, indicating a critical need for robust security practices during the Docker build process. *Dependency error* can lead to multiple symptoms, most notably build failure (107/110=97.27%). This indicates proper dependency management is helpful to build success. *Resource error* is strongly linked to build failure with a ratio of 73.95% (88/119), suggesting that efficient resource allocation and management are pivotal for successful builds. *Execution error* also has build failure as a prominent symptom (64/69=92.75%), highlighting the importance of error-free command execution within the Docker builds. Besides, *Configuration error* has a wide range of symptoms but initialization failure stands out the most ratio of 72.73% (32/44), which implies that correct configuration is essential for a smooth initialization process. *Environment error* is also mostly associated with initialization failure (24/51=47.06%). It shows the sensitivity of initialization failures to build environment settings. These results underscore the importance of addressing these specific root causes to mitigate their associated symptoms.

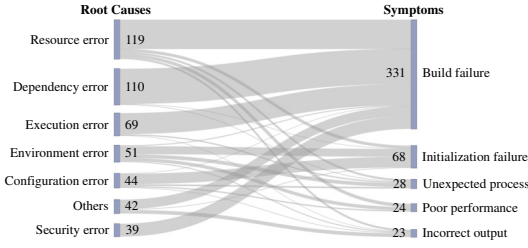


Fig. 6. Mapping of root causes to symptoms.

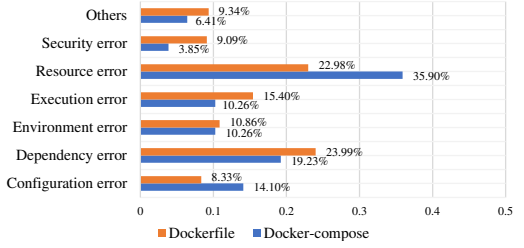


Fig. 7. Comparison of root cause distribution.

Finding 4. Build failure is a common and critical symptom across various root causes, with dependency, execution, and resource errors emerging as particularly impactful root causes (78.26% of build failures). In addition, the results show the sensitivity of Docker build initialization to environmental and configuration factors.

Dockerfile builds vs. Docker-compose builds. We also investigate the root cause distributions in Dockerfile builds and Docker-compose builds. Fig. 7 shows that resource and dependency errors are the two most important root causes in both types of builds. Moreover, *Resource error* (35.90%) is the primary cause of Docker-compose build faults. This type of build often requires the management of multiple services [Jangla 2018], which can lead to DBF if any of the services is not allocated the necessary resources. Dockerfile build faults are primarily caused by *Dependency error* (23.99%). Each command in the Dockerfile can introduce a dependency (e.g., base images, packages, or external resources). Any errors of these dependencies can lead to DBFs. Docker-compose builds also show a higher rate of configuration errors (14.10%) than Dockerfile builds (8.33%). This indicates that DBFs for multi-container applications are more susceptible to configuration errors, which may stem from the complexity of managing multiple services and their inter-dependencies. Execution errors are more frequent in Dockerfile builds (15.40%) than in Docker-compose builds (10.26%). Dockerfile builds proceed in a linear sequence of commands [Anderson 2015]. This sequential nature may make the build process more susceptible to execution errors, which needs to be analyzed in future work. Also, we observe that the ratio of *Others* is slightly higher for Dockerfile builds, indicating that Dockerfile DBFs may be caused by a broader range of miscellaneous issues.

Finding 5. The main root cause for Docker-compose build faults is *Resource error* (35.90%), while Dockerfile build faults are mainly caused by *Dependency error* (23.99%). Moreover, we find a higher occurrence of DBFs caused by configuration errors in Docker-compose builds, but Dockerfile builds exhibit a higher frequency of faults caused by execution errors.

4.3 RQ3: Fix Patterns

4.3.1 Fix Pattern Summary. We identify 35 common fix patterns with 420 DBFs (see Table 1), which involve 6 major objects (i.e., Environment, Dependency, Configuration, Resource, Tool/plugin, and Security). The number of fix patterns in each object varies from 2 to 9. The remaining 54 (11.39%) of the 474 DBFs have no clear indication about their fix patterns and are fixed by case-by-case methods. Fig. 8 gives some examples of different fix patterns.

Environment. 81 (19.29%) of the faults are fixed via resolving the environment errors. Specifically, changing the base image and container architecture (e.g., *arm64* and *amd64*) both solve 16 faults. E.g., in Example 1, the developer addresses P_{47} 's dependency error by using a new base image (from *dotnet* to *aspnetcore*) with the SDK installed. And I_{219} 's environment error is fixed by changing the Dockerfile with the *arm64* architecture (Example 2). 15 faults can be resolved by modifying the network setting (e.g., IP address and proxy/gateway options). E.g., the connection exception of

Table 1. Common fix patterns for DBFs.

Object	Fix pattern	#DBFs	%DBFs
Environment (81, 19.29%)	Change base image	16	3.81%
	Change container architecture	16	3.81%
	Modify network setting	15	3.57%
	Upgrade Docker version	14	3.33%
	Change OS environment	5	1.19%
	Increase memory	5	1.19%
	Restart Docker daemon	4	0.95%
	Restart build operation	4	0.95%
	Reinstall runtime	2	0.48%
Dependency (106, 25.24%)	Upgrade dependency version	51	12.14%
	Add missing dependency	33	7.86%
	Downgrade dependency version	9	2.14%
	Resolve upstream error	8	1.90%
	Use compatible dependency	5	1.19%
Configuration (71, 16.90%)	Modify build argument	21	5.00%
	Use suitable command	14	3.33%
	Change command parameter	11	2.62%
	Add build argument	9	2.14%
	Adjust command format	8	1.90%
	Modify code/script logic	5	1.19%
	Remove improper command/step	3	0.71%
Resource (89, 21.19%)	Change path/location	37	8.81%
	Set correct working directory	20	4.76%
	Add missing resource	11	2.62%
	Delete redundant files	8	1.90%
	Use .dockerignore	5	1.19%
	Change file name	4	0.95%
	Mount correct resource	4	0.95%
Tool/plugin (54, 12.86%)	Modify tool/plugin setting	33	7.86%
	Use suitable tool/plugin	21	5.00%
Security (19, 4.52%)	Modify authentication setting	5	1.19%
	Add ssh setting	4	0.95%
	Change user permission	4	0.95%
	Add missing permission	3	0.71%
	Change file/directory permission	3	0.71%

I_{70} is fixed by pointing *redis* from “localhost:6379” to “redis:6379”. Upgrading the Docker version fixes 14 faults, e.g., the permission error of I_{144} is fixed by upgrading the Docker from “19.03.6” to “20.10.8”. Besides, there are 8 faults that can be fixed by simply restarting the Docker daemon or build operation. Another 12 faults are fixed by changing the OS environment (e.g., Linux and Windows), increasing memory, and reinstalling runtime.

Dependency. 106 (25.24%) faults are fixed by modifying the dependency. Specifically, upgrading the dependency version resolves 51 faults. E.g., in Example 3, P_{130} ’s dependency error is fixed by updating the *TypeScript* version from “v3.7.2” to “v3.8”. 33 faults are fixed by adding missing dependency. E.g., P_{115} ’s dependency error is fixed by adding “*RUN gem install bundler*” before invoking *bundler*. Downgrading the dependency version can fix 9 faults, e.g., I_{201} ’s dependency error is fixed by changing the *swift* version from “latest” to “5.6.3”. 8 faults are fixed by resolving upstream errors. E.g., the resource error of DF post P_{217} is caused by file inaccessibility error and is fixed after turning on the upstream *apparmor* (a security application for Linux system). Another 5 faults are fixed by using a compatible dependency.

Configuration. There are 7 patterns that act on the configuration, which resolve 71 faults (16.90%). Specifically, developers fix 21 faults by modifying the build argument, e.g., in Example 4, I_{250} ’s resource error is fixed by removing the “- - builder=default” argument. Developers fix 14 faults by using suitable commands, e.g., P_8 ’s fault is fixed by running “*sudo apt clean*” command. Besides, 11 faults are fixed by changing the command parameter. E.g., the file inaccessibility error of P_{152} is fixed by replacing “*RUN ["/bin/bash"]*” by “*RUN ["/bin/sh"]*”. Another 25 faults can be fixed by adding build arguments (e.g., adding “- - no-cache” option in P_{207}), adjusting command format (e.g., adding an ‘&&’ to link RUN commands together in P_{144}), modifying code/script logic (e.g.,

Example 1: 🐛 P ₄₇ Fault description: <i>"dotnet aspnetcore docker build fails with a 145 error code"</i> Symptom: Build failure Root cause: Dependency error Fix pattern: Change base image Fix method: - FROM microsoft/dotnet:latest + FROM microsoft/aspnetcore:1.0.1	Example 2: 🐛 I ₂₁₉ Fault description: <i>"Docker build succeeds, but outputs directory empty"</i> Symptom: Incorrect output Root cause: Environment error Fix pattern: Change container architecture Fix method: Changing the Docker architecture with <code>'--add-architecture arm64'</code> .
Example 3: 🐛 P ₁₃₀ Fault description: <i>"npm run build failing because of TypeScript error in docker build"</i> Symptom: Build failure Root cause: Dependency error Fix pattern: Upgrade dependency version Fix method: Updating local TypeScript version from 'v3.7.2' to at least 'v3.8'.	Example 4: 🐛 I ₂₅₀ Fault description: <i>"exit status 1: ERROR: use 'docker --context=default buildx'..."</i> Symptom: Initialization failure Root cause: Resource error Fix pattern: Modify build argument Fix method: Removing <code>'--builder=default'</code> from default args when <code>'use=buildx'</code> .
Example 5: 🐛 P ₅ Fault description: <i>"Unable to copy file during docker build"</i> Symptom: Build failure Root cause: Resource error Fix pattern: Change path/location Fix method: Moving the <code>'requirements.txt'</code> into the <code>'backend'</code> directory.	Example 6: 🐛 P ₁₉ Fault description: <i>"Run dpkg command in docker build in jenkins pipeline throws 'cannot access archive: No such file or directory'..."</i> Symptom: Build failure Root cause: Resource error Fix pattern: Set correct working directory Fix method: + WORKDIR /tmp
Example 7: 🐛 I ₇₃ Fault description: <i>"[BUG] Docker build stuck at final 1/2"</i> Symptom: Unexpected process Root cause: Configuration error Fix pattern: Modify tool/plugin setting Fix method: Disabling docker buildkit tool setting: <code>DOCKER_BUILDKIT=0</code> docker build...	Example 8: 🐛 P ₅₂ Fault description: <i>"ERROR: ... do not currently have an active account selected"</i> Symptom: Build failure Root cause: Security error Fix pattern: Modify authentication setting Fix method: Removing the credential helpers from the <code>'docker/config.json'</code> .

Fig. 8. Examples of different fix patterns.

customizing the script behavior to only perform some logic if a certain set of files changed in P₁₂₇), and removing improper command/step (e.g., removing the `'prune'` command in P₁₆₂).

Resource. 89 faults (21.19%) are resolved by modifying the resource setting. For this object, there are 7 fix patterns. In particular, developers resolve 37 faults by changing file path/location. E.g., in Example 5, the resource error of P₅ is fixed by moving the `requirements.txt` into the correct directory. Developers resolve 39 faults by setting the correct working directory (e.g., in Example 6, P₁₉'s fault is fixed by changing the working directory to `"/tmp"`), adding missing resource (e.g., P₃₈'s fault is fixed by copying the `go.mod` and `go.sum` files into the container), and deleting redundant files (e.g., P₁₉₀'s fault is fixed by deleting temporary files to avoid writing them into the image layer). The remaining 3 fix patterns resolve 13 faults, including using `.dockerignore`, changing file name, and mounting correct resource.

Tool/plugin. 54 faults (12.86%) are resolved by fixing tool/plugin related errors. Modifying tool/plugin settings and using a suitable tool/plugin can fix 33 and 21 faults, respectively. E.g., in Example 7, the configuration error of I₇₃ is fixed by disabling the docker Buildkit tool setting as `"DOCKER_BUILDKIT=0"` before the build command. The file inaccessibility error of I₁₁₉ is fixed by using the `btrfs` storage driver.

Security. Developers fix 19 faults (4.52%) by resolving security issues. Specifically, developers fix 5 faults by modifying authentication settings. E.g., in Example 8, the security error of P₅₂ is fixed by removing the credential from the `.docker/config.json`. 7 faults are fixed by changing user/file/directory permission. Another 7 faults are fixed by adding ssh settings and missing permission.

Finding 6. We summarize 35 common fix patterns based on 6 objects, covering 88.61% of DBFs in this study. More than half (65.72%) of the DBFs are fixed by patterns related to dependency, resource, and environment. The top two frequent fix patterns are upgrading the dependency version (12.14%) and correcting path/location (8.81%).

4.3.2 Fix Pattern Distribution. We further analyze the distribution of fix patterns for different root causes. In Fig. 9, each cell shows the number of DBFs caused by a particular root cause and fixed by a particular fix pattern⁵. We observe complicated crossovers between different fix patterns and root causes, proving that DBF resolution is a challenging problem. Specifically, except for the *Others*, all root causes can be fixed by at least 7 fix patterns. And all frequent fix patterns can be utilized in fixing DBFs that are caused by execution error for at least once.

We find that although the relationship between root causes and fix patterns is complex, there are prevalent patterns that can fix multiple root causes. Specifically, we observe that using suitable command is used in mitigating 7 root causes, and is involved in the fix for 14 faults. The challenge is to decide which command to use given a DBF context. Besides, upgrading the Docker version, upgrading the dependency version, and modifying tool/plugin setting are the other three fix patterns spanning 6 root causes. We also observe that there exist frequent pairs of fix patterns and root cause categories, which indicates that given a root cause category, most faults in that category can be fixed by a specific fix pattern. In particular, we find that for 3 root cause categories (i.e., Configuration, Dependency, and Resource errors), 6 specific fix patterns can fix more than 10 DBFs. For example, upgrading the dependency version and adding missing dependency mainly tackle the *Dependency error*, and 57.14% of faults in this category can be fixed by them. While these two fix patterns are simple, the key challenge is to determine which dependency or dependency version to use for addressing a specific DBF. Correcting path/location has a strong correlation to the root causes of *Resource error*, and 30.23% of faults in this category can be fixed by it.

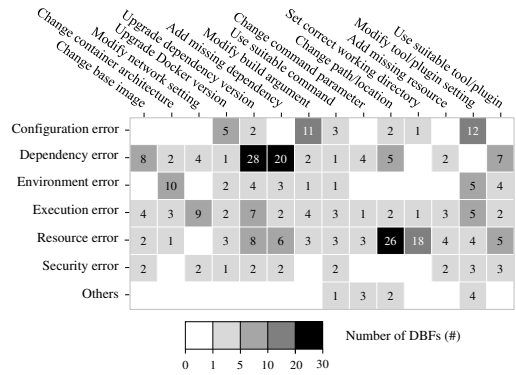


Fig. 9. Distribution of fix patterns for root causes.

Finding 7. We find that faults caused by the same category of root causes can be fixed by diverse patterns in practice. Also, there exist prevalent fix patterns and frequent pairs to address faults in multiple and single root cause categories, respectively.

4.4 Additional Analysis

We further validate the taxonomies of our study by analyzing a new dataset of DBFs. Specifically, we obtain 52 new samples (including 38 GH issues, 13 SO posts, and 1 DF post) created between January 1, 2024 and April 30, 2024 through the data collection methods described in Sec. 3.1. Then we inspect the 52 samples by following the similar analysis methods of our RQs. Our goal is to assess if these samples introduce new symptoms, root causes, or fix patterns that can not be covered by our taxonomies. Through the analysis, we find that these samples' symptoms, root causes, and fix patterns are all covered by our taxonomies: (1) For the symptoms, these samples belong to 5 inner categories and 8 out of 15 (53.33%) leaf categories in our symptom taxonomy. Build failure is

⁵Due to space limit, fix patterns with total frequency < 10 in Table 1 are not shown in the figure.

still the most common symptom (44/52=84.62%). (2) For the root causes, these samples belong to 6 inner categories and 16 out of 23 (69.57%) leaf categories in our root cause taxonomy. Dependency error is still one of the most root causes (15/52=28.85%). (3) For the fix patterns, these samples belong to 6 objects and 23 out of 35 (65.71%) fix patterns. Modifying the dependency still fixes most DBFs, covering 25% (13/52) of the samples. The detailed labeling results of 52 samples can be found in our artifact package [DBF-Artifact 2024]. To sum up, the analysis confirms that our original taxonomies comprehensively cover symptoms, root causes, and fix patterns in the new dataset, with no new categories emerging post-study.

5 Implications and Application

We discuss the implications of our study for different stakeholders, and develop a knowledge-based application for DBF understanding and fixing.

5.1 Implications

The results of our study can not only help Docker developers and their Docker build practice, but also help researchers and service/tool providers to better decide where to focus their efforts.

For developers. Based on our findings, there are several suggestions for developers to optimize their Docker build practices:

(1) **Performing proactive prevention of symptoms at an early stage.** Our study provides developers with critical insights into the common DBF symptoms (RQ1), reminding them to prevent potential DBFs as early as possible. By being cognizant of these symptoms, developers can implement proactive methods to mitigate risks. For example, developers should develop a robust testing strategy that includes automated tests to catch code failures before they propagate through the Docker build process. They can utilize CI tools (e.g., GitHub Actions [Decan et al. 2022]) to run tests and provide immediate feedback. In addition, developers should ensure that all environment factors are correctly initialized before the Docker build process begins, e.g., implementing health checks and readiness probes to verify the state of the environment.

(2) **Troubleshooting DBFs with appropriate and efficient fixes.** Our study uncovers the common root causes of DBFs (RQ2), such as resource and dependency errors, which account for a substantial portion of DBFs (48.32%). By understanding these root causes, developers can implement more precise and effective strategies for diagnosing, localizing, and resolving DBFs. For example, developers should define a clear and minimal build context, avoiding the inclusion of unnecessary or problematic files. Also, developers should use tools like GitHub *dependabot* [He et al. 2023] to keep track of updates and maintain dependency compatibility. Moreover, our study identifies the common fix patterns that have been successful in addressing DBFs (RQ3). Developers can use our fix patterns as guidance for both manual and automated DBF resolution.

(3) **Paying more attention to resources and configurations in Docker-compose builds.** Our study finds that Docker-compose builds have higher percentage of DBFs caused by resource error and configuration error than Dockerfile builds (RQ2). Thus, developers should focus on resource allocation and configuration settings in Docker-compose builds. They should familiarize themselves with the syntax and best practices of the *docker-compose.yml* file, especially how to properly allocate resources and configure dependencies between services. Inexperienced developers are encouraged to seek out targeted learning resources (e.g., official Docker documentation [Docker 2024d]) before implementing Docker-compose builds. This skill enhancement will bolster their ability to anticipate and effectively resolve faults in Docker-compose builds.

For researchers. Our findings provide future research implications in three directions:

(1) **DBF detection techniques are needed.** Our study indicates that only 14.35% of DBFs are introduced in the initialization stage, while 85.65% of DBFs are discovered during or after the

Docker build execution (**RQ1**). Thus, it may indicate that many DBFs stay undetected until later build lifecycle stages. To detect DBFs as early as possible, researchers can leverage historical build data to train machine learning models that can predict the likelihood of a DBF based on early indicators (e.g., code changes, dependency updates, and configuration alterations). Also, researchers can develop anomaly detection techniques to identify deviations from normal build behavior, which could signal an impending DBF.

(2) **DBF localization techniques are needed.** Our study reveals that a particular symptom of DBF may be caused by different root causes (**RQ2**), and the objects of fix patterns that repair the root cause may also be different (**RQ3**). Therefore, it is often challenging to localize DBFs. During our manual analysis, we find that developers often use logs as the clue to locate DBFs. Hence, automated log analysis techniques should be developed to parse log statements and locate potential DBFs using log traces. Once a DBF is localized, automated fixing techniques can use the fix patterns derived from our study to fix it.

(3) **DBFs in multi-container build scenarios need further analysis.** Our research reveals differences in symptoms and root causes between Dockerfile builds and Docker-compose builds (**RQ1** and **RQ2**). Researchers should focus on examining the reasons for these disparities in multi-container build scenarios and provide advanced techniques for addressing DBFs within such complex environments. For example, the higher incidence of initialization failure and incorrect output in these builds suggests a need for research into effective orchestration techniques that ensure all services within the multi-container architecture are correctly initialized and function harmoniously. Moreover, researchers should investigate scalable solutions and intelligent resource management to address the resource and configuration errors that are more prevalent in this multi-container build environment.

For service/tool providers. Service/tool providers are well-positioned to leverage our insights to enhance their offerings. Our study finds that developers encounter many problems in understanding and maintaining Docker build configuration (**RQ1** and **RQ2**). Service providers should consider *offering easy-to-understand educational resources and best practice guides* that help developers understand and mitigate the common issues related to Docker build configuration. Our study also reveals that poor performance is a typical symptom for DBFs (**RQ1**). Thus, tool providers should *offer performance profiling tools* that help developers understand and optimize the performance of their Docker builds. Due to the reliance on human participation and lack of sharing of knowledge fragments, current Q&A forums and issue tracking systems may take a long time for developers to get valuable responses or to resolve DBFs. For instance, our statistics on the DBF dataset find that the average time for a SO post to get an accepted answer is 528.05 hours (median: 9.9 hours), and the average time for a GH issue to get closed is 90.9 days (mean: 8.6 days). This highlights the need for *advanced tools that can help developers quickly find and learn dispersed knowledge*, thus enhancing the way developers search, recognize, and overcome DBFs.

5.2 Application

Our study has accumulated dispersed knowledge from multiple data sources, offering rich insights that can be harnessed to guide subsequent operations. Thus, we envision an automated tool that can act as a knowledgeable companion, standing ready to assist developers with the accumulated knowledge. It can provide immediate feedback to developers, which not only alleviates the cognitive load on developers (especially inexperienced novices) but also potentially enhances the efficiency and effectiveness of DBF resolution. We implement the above idea as a knowledge-based application named DBF-ASSISTANT to assist developers in understanding and fixing DBFs.

Design and implementation. At the high level, DBF-ASSISTANT leverages the Large Language Model (LLM) and Retrieval Augmented Generation (RAG) [Lewis et al. 2020] to incorporate DBF

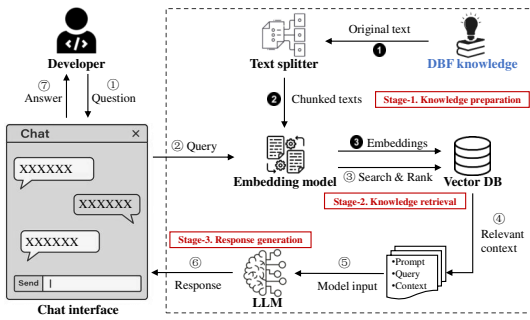


Fig. 10. Workflow of DBF-ASSISTANT.

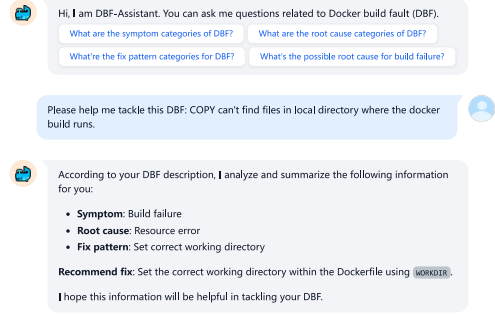


Fig. 11. Example of interaction with DBF-ASSISTANT.

knowledge to better answer developers' questions about DBFs. LLM has ability to understand complex query and generate informative response in Q&A scenarios. RAG extends the already powerful capabilities of LLMs to specific domain's knowledge base, all without the need to retrain the model. To construct the DBF knowledge, we collect three types of documents: (1) *Empirical findings*: We summarize the derived DBF symptoms, root causes, and fix patterns into a DOC document, including their categories and brief descriptions; (2) *DBF dataset*: We deposit the annotated 474 DBFs data into an XLSX document, including the basic information of each issue or post, and corresponding symptom, root cause, and fix pattern labels; and (3) *Docker official documents*: We collect several HTML pages about Docker build in Docker's official website, e.g., Docker build overview [Docker 2024c] and Build context [Docker 2024a]. These knowledge documents are available in our artifact package [DBF-Artifact 2024].

Fig. 10 depicts the workflow of DBF-ASSISTANT. Specifically, DBF-ASSISTANT involves the interaction with developers through a *chat interface* (i.e., front-end module), which ensures that the interaction is conversational and user-friendly. DBF-ASSISTANT's back-end module engages in a multi-stage process to provide a valuable answer: (1) *Stage-1. Knowledge preparation*: DBF-ASSISTANT utilizes a *Text splitter* to break down and process the original DBF knowledge documents into smaller, more manageable chunks. Then, these chunks are subsequently converted into embeddings and stored in a vector database. This preparation is essential for efficient knowledge retrieval and interaction. Note that this stage is operated only once at the beginning and the knowledge can be updated subsequently. (2) *Stage-2. Knowledge retrieval*: After a query being initiated, DBF-ASSISTANT uses the *Embedding model* and *Vector DB* to interpret the query, semantically search for relevant context within the knowledge base, and rank contexts based on their relevance to the input query. (3) *Stage-3. Response generation*: Once relevant context is identified, DBF-ASSISTANT formulates a model input that combines system prompt, query, and retrieved context. The model input is then processed through a pre-trained LLM, providing LLM with a more comprehensive understanding of the domain-specific knowledge. Then, LLM uses its in-context learning abilities to generate a coherent and contextually appropriate response. Finally, the response is sent back to the developer through the chat interface.

There are two main application scenarios for which DBF-ASSISTANT can provide assistive support: (1) *Scenario-1: Basic knowledge acquisition*: It allows developers to query and learn knowledge related to symptoms, root causes and fix patterns of DBFs, e.g., *what are the main symptom categories of DBF? what are the possible root causes of build failure?* This helps developers to quickly understand the basic concepts about DBF and thus prepares them to tackle the complexities inherent in the subsequent problems. (2) *Scenario-2: DBF identification, localization, and fix recommendation*: It supports the prediction of symptom, root cause, and fix pattern types based on developer-supplied

Table 2. Results of identification performance.

Metric	Human	DBF-Assistant w/o knowledge	DBF-Assistant
Hit rate (Symptom)	71.43%	28.57%	78.57%
Hit rate (Root cause)	50.00%	42.86%	71.43%
Hit rate (Fix pattern)	35.71%	7.14%	50.00%
Average latency	2,233.44s	31.35s	76.44s

build fault descriptions, and can further recommend fixes. Fig. 11 gives an example of interaction with DBF-ASSISTANT. This allows developers to obtain valuable community knowledge to discover and resolve DBFs more efficiently.

In our implementation, DBF-ASSISTANT uses Nomic Embed model [Nussbaum et al. 2024] to generate embeddings, and uses Google’s Gemma 2 model [Team et al. 2024] to learn knowledge and generate response. Those two models are open sourced and have strong capabilities in text embedding and exploration. Certainly, they can be replaced by other powerful models in the future. DBF-ASSISTANT is deployed and runs on a 64-bit CentOS 7.6 server with 8-core CPUs and 32 GB memory (without GPU).

Effectiveness evaluation. To evaluate DBF-ASSISTANT’s effectiveness, we focus on the identification performance of *Scenario-2* and conduct experiments based on the 14 post samples from the additional analysis (they are not in the DBF dataset). We organize the title and description of each post sample into query, input it to DBF-ASSISTANT, and evaluate the generated response. We do not consider those issue samples because their problem descriptions are not as self-contained as posts’, and their resolution depends on the project-specific knowledge. We use two evaluation metrics, i.e., *hit rate* and *response latency*. The first metric is the percentage of generated category labels (symptom, root cause, and fix pattern) equal to the ground-truth category labels summarized in Sec. 4.4. The second metric is the consumed time on finishing each response. We also add two baselines: (1) *DBF-ASSISTANT w/o knowledge*: We remove DBF knowledge from DBF-ASSISTANT (i.e., RAG function is disabled) and then rerun the experiment to understand the impact of DBF knowledge on the application. (2) *human study*: We recruit 7 participants to manually analyze the 14 examples. These participants are graduate students who are familiar with Docker, and are recruited voluntarily at our college. All participants have worked on at least one or two research projects that utilize Docker and have hands-on experience on Docker build. Each participant is given 2 examples and is asked to identify the DBF category information, as well as write a short description of the fix method with their expertise. This human study aims to investigate how much effort can be saved for developers.

Table 2 shows the result of identification performance. DBF-ASSISTANT achieves hit rates in the symptom, root cause, and fix pattern categories of 78.57%, 71.43%, and 50.00%, respectively. We find that DBF-ASSISTANT’s hit rates decrease when DBF knowledge is removed. For example, DBF-ASSISTANT correctly hit the symptom type for 11 out of 14 samples, and the number of hits decreased to 4 samples after removing DBF knowledge. In terms of response latency, DBF-ASSISTANT costs averagely 76.44 seconds, which is averagely 2.4 times longer than DBF-ASSISTANT without DBF knowledge (31.35 seconds). This is due to increased time for knowledge base queries. Therefore, DBF knowledge is important to improve the performance of DBF-Assistant but introduces additional query time. When comparing DBF-ASSISTANT and human study, we find that the hit rates of human analysis are lower than DBF-ASSISTANT, and the human analysis takes averagely 29.2 times longer than DBF-ASSISTANT. Thus, despite the extra query time, DBF-ASSISTANT can be useful for developers to provide accurate DBF identification and greatly save resolution time.

Furthermore, we conduct a preliminary evaluation on the user satisfaction. We ask the same 7 participants to use DBF-ASSISTANT for one week to ask any interested DBF-related questions, and to rate the responses generated by DBF-ASSISTANT as “like” or “dislike”. In one week, we receive

125 interactions, of which 94 (75.2%) are marked with “like”, and the remaining 31 are marked with dislike. We ask participants about why they “dislike”. Participant feedback consists of two main reasons: (1) DBF-ASSISTANT does not produce the expected results and (2) DBF-ASSISTANT is slow to respond. Our further inspection reveals that many of the unintended results are due to bad question formulation. Overall, DBF-ASSISTANT shows promise in assisting with DBF resolution.

We know that there are still many directions in which DBF-ASSISTANT can be enhanced. For example, we find that DBF-ASSISTANT’s hit rate decreases a lot when predicting fix patterns. This may be because the same type of root cause can be addressed by different fix patterns, as we found in Sec. 4.3.2. It can be enhanced by increasing the amount of annotated data in the knowledge base. Also, integrating project-specific knowledge (e.g., source code files), using more powerful LLM, and upgrading hardware configurations (e.g., use GPUs) can help improve the effectiveness of the DBF-ASSISTANT. Furthermore, increasing the experimental sample size and comparing DBF-ASSISTANT with commercial LLMs (e.g., ChatGPT) can help strengthen the outcome of evaluation. We leave such optimizations and evaluation as future work.

6 Threats to Validity

The threats to the validity of the study can be divided into two types: (1) The internal threat lies in our manual labeling process. To minimize the possible subjectivity of researchers, we ensure that each data item is separately labeled by two authors and a third arbiter is used to resolve conflicts. Thus, we believe that we reduce labeling bias to an acceptable minimum. To ensure consistency and soundness of classification, we use Cohen’s kappa coefficient [Cohen 1960] and find high inter-rater agreement among the annotators. We also send the taxonomies to the recruited 7 participants (in Sec. 5.2) as well as to some experienced developers and researchers we know, and they express agreement with our classification results. (2) The external threat lies in the dataset. Our study cannot cover all real-world DBFs. To mitigate this threat, we select three representative data sources (i.e., GH, SO, and DF), where GH and SO have been widely used in previous empirical studies [Bagherzadeh and Khatchadourian 2019; Bajaj et al. 2014; Haque et al. 2020; Rosen and Shihab 2016], and DF is the official forum of Docker. Moreover, we conduct additional investigation on another 52 DBFs that are not in our DBF dataset, which demonstrates that no new categories have been introduced (Sec. 4.4). However, there could be other DBFs that are not covered in our study. For example, developers may communicate their discussions in other media (e.g., mailing lists). Future study considering other data sources may further complement our study.

7 Related Work

In recent years, there has been a significant focus on understanding build faults in different types of software systems. Hassan et al. [2017] conducted a study on 91 build problems of 200 Java projects, proposing a taxonomy of build problems. Vassallo et al. [2017] compared build failure occurrences between 349 open-source Java projects and 418 projects from ING Nederland, creating a taxonomy of build failures. Sulír and Porubán [2016] investigated the builds in Java projects and found that more than 38% of these projects failed to build. McIntosh et al. [2012] studied the evolution of Java build systems and found that Java builds consistently grow longer as a project ages. Lou et al. [2020] studied 465 build issues of three popular build systems Maven, Ant, and Gradle from SO, and constructed a fine-granularity taxonomy regarding to the failure symptoms. These studies motivate the importance of build management. Our work is inspired by them to systematically characterize DBFs across Docker-based projects.

Recently, some relevant studies have been conducted and shed some light on the DBFs. Table 3 lists the summary of those studies involving DBFs. Specifically, Cito et al. [2017] examined GitHub Dockerfile data and found 2 symptoms in Dockerfile builds, i.e., high failure rate and long duration.

Table 3. Summary of prior relevant studies involving DBFs.

Relevant study	Build type	Data source	Symptoms	Root causes	Fix patterns	DBF-related results
Cito et al. [2017]	Dockerfile	GH	?	×	×	2 symptoms (build failure and long duration)
Wu et al. [2020]	Dockerfile	GH	?	×	×	1 symptom (build failure)
Haque et al. [2020]	Unknown	SO	?	×	×	2 symptoms (coding issues and networking error)
Henkel et al. [2021]	Dockerfile	GH	?	?	×	1 symptom (build failure) and 4 root causes
Wu et al. [2022]	Dockerfile	GH	?	?	×	1 symptom (slow build) and 3 root causes
Our work	Dockerfile Docker-compose	GH, SO, DF	✓	✓	✓	15 symptoms, 23 root causes, and 35 fix patterns

"✓", "×", "?" denote that the study does, does not or only partially does the investigation, respectively. GH: GitHub, SO: Stack Overflow, and DF: Docker Forum.

Wu et al. [2020] presented an empirical study of Docker build failure and found that the overall build failure rate in the Docker context is 17.8%. Haque et al. [2020] analyzed SO posts and derived challenge topics faced by developers with respect to Docker technology; 2 of their derived topics (i.e., coding issues and networking error) are related to DBF symptoms. Henkel et al. [2021] studied Dockerfiles on GitHub, and found that over a quarter of the examined Dockerfiles failed to build. They also proposed 4 root causes, including external file changes, inconsistent version dependency, missing commands, and project-specific failures. Wu et al. [2022] investigated Docker builds from GitHub projects and found that slow build has been a shared barrier confronted by developers. They summarized 3 root causes, i.e., high build complexity, unsuitable context churn, and inappropriate configuration. Despite these efforts, the characteristics of DBFs (i.e., symptoms, root causes, and fix patterns) are still under-investigated and thus we aim to fill in this knowledge gap.

Overall, our study differs from prior DBF-related studies in terms of *target* and *diversity*: (1) Prior works focused on the discussion topics of broader Docker technology, Dockerfile usage, or statistics on build failure and build duration. Although they mentioned a few DBF symptoms and root causes, they did not analyze how developers fixed them in practice. Different from them, our study aims to provide a comprehensive understanding of the real-world DBFs, including their symptoms, root causes, and fix patterns. (2) Prior works only partially analyzed one or two types of DBF, leaving many other categories of DBFs unexplored. Our taxonomies cover extra categories that are absent in previous studies. Different from prior works conducted on a single source of data, we select three representative data sources to collect the broader DBF knowledge of the Docker community. Also, prior works lacked of the analysis of Docker-compose (an important artifact for multi-container applications), whereas we analyze both Dockerfile builds and Docker-compose builds.

8 Conclusion

In this work, we present the first comprehensive study on DBFs by manually inspecting 255 GH issues and 219 SO&DF posts. We first construct taxonomies for fault symptoms, root causes, and fix patterns, respectively. We find that DBFs stretch over a wide spectrum of symptoms and root causes, and prevalent fix patterns can be utilized to facilitate DBF resolution. Besides, we also analyze the different fault distribution between Dockerfile builds and Docker-compose builds. Based on our findings, we provide a series of implications for researchers, developers, and service/tool providers, and develop a knowledge-based application DBF-ASSISTANT, to assist developers on build management in engineering Docker-based applications.

Data Availability

The data and code of our study are available at <https://sites.google.com/view/dbf-research>.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No.62202480).

References

- Charles Anderson. 2015. Docker [software engineering]. *IEEE Software* 32, 3 (2015), 102–c3. doi:10.1109/MS.2015.62
- Brandon Atkinson and Dallas Edwards. 2018. *Generic Pipelines Using Docker: The DevOps Guide to Building Reusable, Platform Agnostic CI/CD Frameworks*. Apress. doi:10.1007/978-1-4842-3655-0
- M. Bagherzadeh and R. Khatchadourian. 2019. Going Big: a Large-Scale Study on What Big Data Developers Ask. In *Proceedings of the European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 432–442. doi:10.1145/3338906.3338939
- K. Bajaj, K. Pattabiraman, and A. Mesbah. 2014. Mining questions asked by web developers. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*. 112–121. doi:10.1145/2597073.2597083
- David Bernstein. 2014. Containers and cloud: From lxc to docker to kubernetes. *IEEE Cloud Computing* 1, 3 (2014), 81–84. doi:10.1109/MCC.2014.51
- Carl Boettiger. 2015. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review* 49, 1 (2015), 71–79. doi:10.1145/2723872.2723882
- Noah Bühlmann and Mohammad Ghafari. 2022. How do developers deal with security issue reports on github?. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*. 1580–1589. doi:10.1145/3477314.3507123
- Jürgen Cito, Vincenzo Ferme, and Harald C Gall. 2016. Using docker containers to improve reproducibility in software and web engineering research. In *Proceedings of International Conference on Web Engineering (ICWE)*. Springer, 609–612. doi:10.1145/2889160.2891057
- Jürgen Cito, Gerald Schermann, John Erik Wittern, Philipp Leitner, Sali Zumberi, and Harald C Gall. 2017. An empirical analysis of the docker container ecosystem on github. In *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 323–333. doi:10.1109/MSR.2017.67
- Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- Theo Combe, Antony Martin, and Roberto Di Pietro. 2016. To docker or not to docker: A security perspective. *IEEE Cloud Computing* 3, 5 (2016), 54–62. doi:10.1109/MCC.2016.100
- DBF-Artifact. 2024. Artifact Package for DBF Study. [online]. <https://sites.google.com/view/dbf-research>.
- Alexandre Decan, Tom Mens, Pooya Rostami Mazrae, and Mehdi Golzadeh. 2022. On the use of github actions in software development repositories. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 235–245. doi:10.1109/ICSME55016.2022.00029
- Docker. 2022a. docker build. [online]. <https://docs.docker.com/engine/reference/commandline/build/>, Accessed September 1, 2024.
- Docker. 2022b. Docker compose. [online]. <https://docs.docker.com/engine/reference/commandline/compose/>, Accessed September 1, 2024.
- Docker. 2024a. Build context. [online]. <https://docs.docker.com/build/concepts/context/>, Accessed September 1, 2024.
- Docker. 2024b. Docker build cache. [online]. <https://docs.docker.com/build/cache/>, Accessed September 1, 2024.
- Docker. 2024c. Docker Build Overview. [online]. <https://docs.docker.com/build/concepts/overview/>, Accessed September 1, 2024.
- Docker. 2024d. Docker compose build. [online]. <https://docs.docker.com/reference/cli/docker/compose/build/>, Accessed September 1, 2024.
- Lian Du, Tianyu Wo, Renyu Yang, and Chunming Hu. 2017. Cider: a rapid docker container deployment system through sharing network storage. In *Proceedings of IEEE 19th International Conference on High Performance Computing and Communications; IEEE 15th International Conference on Smart City; IEEE 3rd International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 332–339. doi:10.1109/HPCC-SmartCity-DSS.2017.44
- Rajdeep Dua, A Reddy Raja, and Dharmesh Kakadia. 2014. Virtualization vs containerization to support paas. In *Proceedings of IEEE International Conference on Cloud Engineering (ICCE)*. IEEE, 610–614. doi:10.1109/IC2E.2014.41
- Kalvin Eng, Abram Hindle, and Eleni Stroulia. 2024. Patterns of multi-container composition for service orchestration with Docker Compose. *Empirical Software Engineering* 29, 3 (2024), 65. doi:10.1007/s10664-024-10462-8
- Somya Garg and Satvik Garg. 2019. Automated cloud infrastructure, continuous integration and continuous delivery using docker with robust container security. In *Proceedings of IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)*. IEEE, 467–470. doi:10.1109/MIPR.2019.00094
- GitHub. 2022. Search issues and pull requests. [online]. <https://docs.github.com/en/rest/search/search?apiVersion=2022-11-28#search-issues-and-pull-requests>, Accessed September 1, 2024.
- Mubin Ul Haque, Leonardo Horn Iwaya, and M Ali Babar. 2020. Challenges in docker development: A large-scale study using stack overflow. In *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11. doi:10.1145/3382494.3410693
- Foyzul Hassan, Shaikh Mostafa, Edmund SL Lam, and Xiaoyin Wang. 2017. Automatic building of java projects in software repositories: A study on feasibility and challenges. In *Proceedings of ACM/IEEE International Symposium on Empirical*

- Software Engineering and Measurement (ESEM)*. IEEE, 38–47. doi:10.1109/ESEM.2017.11
- Runzhi He, Hao He, Yuxia Zhang, and Minghui Zhou. 2023. Automating dependency updates in practice: An exploratory study on github dependabot. *IEEE Transactions on Software Engineering* (2023), 1–18. doi:10.1109/TSE.2023.3278129
- Viktor Hedefalk. 2018. How do I copy variables between stages of multi stage Docker build? [online]. <https://stackoverflow.com/questions/52904847/how-do-i-copy-variables-between-stages-of-multi-stage-docker-build>, Accessed September 1, 2024.
- Jordan Henkel, Denini Silva, Leopoldo Teixeira, Marcelo d’Amorim, and Thomas Reps. 2021. Shipwright: A Human-in-the-Loop System for Dockerfile Repair. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1148–1160. doi:10.1109/ICSE43902.2021.00106
- Md Hasan Ibrahim, Mohammed Sayagh, and Ahmed E Hassan. 2021. A study of how Docker Compose is used to compose multi-component systems. *Empirical Software Engineering* 26 (2021), 1–27. doi:10.1007/s10664-021-10025-1
- Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A comprehensive study on deep learning bug characteristics. In *Proceedings of the 27th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering (ESEC/FSE)*. 510–520. doi:10.1016/j.jksuci.2023.01.014
- Kinnary Jangla. 2018. Docker compose. *Accelerating Development Velocity Using Docker: Docker Across Microservices* (2018), 77–98. doi:10.1007/978-1-4842-3936-0
- Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M German, and Daniela Damian. 2016. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering* 21 (2016), 2035–2071. doi:10.1007/s10664-015-9393-5
- Emna Ksontini, Marouane Kessentini, Thiago do N. Ferreira, and Foyzul Hassan. 2021. Refactorings and Technical Debt in Docker Projects: An Empirical Study. In *the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 781–791. doi:10.1109/ASE51524.2021.9678585
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- Heng Li, Foutse Khomh, Moses Openja, et al. 2021. Understanding quantum software engineering challenges an empirical study on stack exchange forums and github issues. In *Proceedings of IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 343–354. doi:10.1109/ICSME52107.2021.00037
- Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding build issue resolution in practice: symptoms and fix patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 617–628. doi:10.1145/3368089.3409760
- Shane McIntosh, Bram Adams, and Ahmed E Hassan. 2012. The evolution of Java build systems. *Empirical Software Engineering* 17, 4 (2012), 578–608. doi:10.1007/s10664-011-9169-5
- Mcarans Mike. 2020. “Build finished” Docker build hangs on “Canceling”. [online]. <https://forums.docker.com/t/build-finished-docker-build-hangs-on-canceling/90527>, Accessed September 1, 2024.
- Adrian Mouat. 2015. *Using Docker: Developing and deploying software with containers*. " O'Reilly Media, Inc".
- Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. 2024. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv:2402.01613* (2024).
- Luca Ponzanelli, Andrea Mocchi, Alberto Bacchelli, Michele Lanza, and David Fullerton. 2014. Improving low quality stack overflow post detection. In *Proceedings of IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 541–544. doi:10.1109/ICSME.2014.90
- Pprathima3. 2018. Docker build image failing with docker load command failed with output. [online]. <https://forums.docker.com/t/docker-build-image-failing-with-docker-load-command-failed-with-output/64936>, Accessed September 1, 2024.
- Nicolas R. 2017. Extremely slow parallel docker builds. [online]. <https://forums.docker.com/t/extremely-slow-parallel-docker-builds/38587>, Accessed September 1, 2024.
- Christoffer Rosen and Emad Shihab. 2016. What are mobile developers asking about? a large scale study using stack overflow. *Empirical Software Engineering* 21 (2016), 1192–1223. doi:10.1007/s10664-015-9379-3
- Carolyn B. Seaman. 1999. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on software engineering* 25, 4 (1999), 557–572. doi:10.1109/32.799955
- Hyunmin Seo, Caitlin Sadowski, Sebastian Elbaum, Edward Aftandilian, and Robert Bowdidge. 2014. Programmers’ build errors: a case study (at google). In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 724–734. doi:10.1145/2568225.2568255
- Edvard Shalaev. 2021. Containerized Software Development for Industrial Environment. (2021).
- Randall Smith. 2017. *Docker orchestration*. Packt Publishing Ltd.
- Stack-Overflow. 2023. 2023 Developer Survey. [online]. <https://survey.stack-overflow.co/2023/>, Accessed September 1, 2024.
- Stack-Overflow. 2024. Stack Exchange Data Explorer. [online]. <https://data.stackexchange.com/stackoverflow/query/>, Accessed September 1, 2024.

- stefan79. 2015. Ansible Provisioner cannot complete on docker build. [online]. <https://github.com/hashicorp/packer/issues/2028>, Accessed September 1, 2024.
- Matúš Sulír and Jaroslav Porubán. 2016. A quantitative study of java software buildability. In *Proceedings of the 7th International Workshop on Evaluation and Usability of Programming Languages and Tools*. 17–25. doi:10.1145/3001878.3001882
- Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv:2408.00118* (2024).
- Carmin Vassallo, Gerald Schermann, Fiorella Zampetti, Daniele Romano, Philipp Leitner, Andy Zaidman, Massimiliano Di Penta, and Sebastiano Panichella. 2017. A tale of CI build failures: An open source and a financial organization perspective. In *Proceedings of IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 183–193. doi:10.1109/ICSME.2017.67
- Whoibrar. 2024. What Is Docker Daemon? [online]. <https://www.geeksforgeeks.org/what-is-docker-daemon/>, Accessed September 1, 2024.
- Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. 2020. An empirical study of build failures in the docker context. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. 76–80. doi:10.1145/3379597.3387483
- Yiwen Wu, Yang Zhang, Kele Xu, Tao Wang, and Huaimin Wang. 2022. Understanding and Predicting Docker Build Duration: An Empirical Study of Containerized Workflow of OSS Projects. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1–13. doi:10.1145/3551349.3556940
- Tianyin Xu and Darko Marinov. 2018. Mining container image repositories for software configuration and beyond. In *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 49–52. doi:10.1145/3183399.3183403
- Yang Zhang, Bogdan Vasilescu, Huaimin Wang, and Vladimir Filkov. 2018. One size does not fit all: an empirical study of containerized continuous deployment workflows. In *the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 295–306. doi:10.1145/3236024.3236033
- Yang Zhang, Yiwen Wu, Tingting Chen, Tao Wang, Hui Liu, and Huaimin Wang. 2024. How do Developers Talk about GitHub Actions? Evidence from Online Software Development Community. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13. doi:10.1145/3597503.3623327
- Yinyuan Zhang, Yang Zhang, Xinjun Mao, Yiwen Wu, Bo Lin, and Shangwen Wang. 2022. Recommending Base Image for Docker Containers based on Deep Configuration Comprehension. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 449–453. doi:10.1109/SANER53432.2022.00060
- Mahdis Zolfagharinia, Bram Adams, and Yann-Gaël Guéhéneuc. 2019. A study of build inflation in 30 million CPAN builds on 13 Perl versions and 10 operating systems. *Empirical Software Engineering* 24, 6 (2019), 3933–3971. doi:10.1007/s10664-019-09709-6

Received 2024-09-13; accepted 2025-01-14